

Project End-to-End MLOps System

1. Project Context

You will implement a basic MLOps solution for a machine learning regression problem.

You will begin with a data science notebook in which the exploratory analysis and modeling tasks are done. From there, you will progressively migrate the project to be an MLOps repository.

It's important to note that your target here isn't building a full production pipeline, but rather understanding the key concepts behind MLOps:

- Data extraction
 - Data validation
 - Data cleaning
 - Model training
 - MLflow experiment tracking
 - Model Registry and champion model selection
 - Model serving with an API
 - Basic error handling
 - CI/CD automation with GitHub Actions
 - Clear project structure and separation of responsibilities
-

2. Suggested Use Case

You can use a simple regression dataset such as:

[Diabetes Dataset](#) from scikit-learn

The goal is to predict disease progression using patient-related numerical features.

You may also choose another simple regression dataset, but it must allow you to clearly demonstrate data preparation, model training, prediction, and evaluation.

3. Project Goal

The final goal is to create a GitHub repository that simulates a basic MLOps workflow.

Your system must:

1. Start from an exploratory notebook.
 2. Extract, validate, and clean the data.
 3. Train at least one machine learning model.
 4. Use MLflow to track experiments.
 5. Register a model in MLflow.
 6. Assign a **champion** model.
 7. Serve the champion model through a FastAPI API.
 8. Implement basic API error handling.
 9. Automate the pipeline with GitHub Actions.
 10. Document the project clearly in a README.
-

4. Repository Requirements

Your repository must be organized in a clear and professional way.

You should separate responsibilities across different folders and files.

You may adapt this structure, but your project must clearly separate:

- Data logic
 - Training logic
 - Model registry/evaluation logic
 - API serving logic
 - Pipeline orchestration
 - CI/CD configuration
-

5. Data Engineering Requirements

Your project must include a basic data engineering flow.

At minimum, you must implement:

Data extraction

Load the dataset and save it in a raw data location.

Data validation

Validate if it is necessary:

- Required columns exist.
- The target column exists.
- Data types are valid.
- There are no critical schema problems.

Data cleaning

Apply simple cleaning logic, such as:

- Handling missing values.
- Ensuring numeric types.
- Removing or treating invalid values if needed.

These steps must be visible first in the notebook and then implemented in reusable scripts inside the repository.

6. Model Training and MLflow Requirements

Your project must use MLflow.

At minimum, you must:

- Train at least one machine learning model.
- Log parameters, metrics, and artifacts with MLflow.
- Register the selected model in the MLflow Model Registry.
- Assign a **champion** alias or equivalent label to the model that passes the quality criteria.

You should define simple evaluation metrics appropriate for your use case.

For a regression use case, examples include:

- RMSE
- MAE
- R^2

You should also define a simple quality gate. For example:

Only assign champion if RMSE is below a defined threshold.

The exact threshold can be chosen by the student, but it must be explained in the README.

7. API Serving Requirements

You must expose the champion model through an API.

The API must include the following endpoints:

GET /health

Used to check whether the service is alive.

Expected behavior:

- Return **200 OK** when the service is running.
- Return a simple JSON response such as:

```
{  
  "status": "ok"  
}
```

GET /model-info

Used to show which model is currently being served.

Expected behavior:

- Return model name.
- Return model version.
- Return champion alias or equivalent.
- Return useful metadata if available.

POST /predict

Used to generate predictions.

Expected behavior:

- Receive input data in JSON format.
- Validate the input schema.
- Return the prediction and model version.

8. Minimum Error Handling Requirements

Your API must handle basic cases correctly.

At minimum:

Successful request

The API must correctly handle valid requests.

- Example:
 - Valid input → 200 OK → prediction returned

Invalid input data

The API must reject badly formed input data.

- Example:
 - Missing feature or wrong data type → 422 validation error or 400 bad request
 - This can be handled using Pydantic models.

Service/model not available

The API must handle the case where the model is not loaded or the service cannot make predictions.

- Example:
 - Model not loaded → 503 Service Unavailable
 - Prediction failure → 500 Internal Server Error

The goal is not to cover every possible production case, but to show that the student understands that APIs must manage both successful and failing scenarios.

9. CI/CD Requirements

Your project must include a GitHub Actions workflow.

The workflow must be stored in:

- .github/workflows/ci.yml
- It must run automatically when code is pushed to **master** or **main**.
- The workflow should execute the main MLOps pipeline in order:
 - data extraction → data validation → data cleaning → model training → model registration/evaluation

The goal is to automate the process so that the model pipeline is executed consistently every time new code is pushed.

10. README Requirements

Your project must include a clear `README.md`.

The README should explain:

- The project goal.
- The dataset used.
- The repository structure.
- How to install dependencies.
- How to run the data pipeline.
- How to train and register the model.
- How the champion model is selected.
- How to run the API.
- How to test `/health`, `/model-info`, and `/predict`.
- How CI/CD works.
- What basic error cases are handled.

It should allow another person to understand and run your project.

12. Final Deliverables

You must deliver a GitHub repository containing:

- Exploratory notebook.
- Data extraction, validation, and cleaning scripts.
- Model training script.
- MLflow tracking and model registry logic.
- Champion model selection.
- FastAPI serving code.
- Endpoints:
 - `GET /health`
 - `GET /model-info`
 - `POST /predict`
- Basic API error handling.
- GitHub Actions workflow.
- README file.
- Requirements file.
- Clear project structure.

13. Evaluation Criteria

Your project will be evaluated based on:

MLOps workflow completeness

The project includes the main stages of the MLOps lifecycle.

Data engineering quality

The project includes extraction, validation, and cleaning steps.

MLflow usage

The project uses MLflow for experiment tracking, model registration, and champion model selection.

API serving

The champion model is available through a working API.

Error handling

The API handles valid requests, invalid input, and unavailable model/service situations.

CI/CD automation

The GitHub Actions workflow runs the pipeline automatically on push.

Code organization

The project separates responsibilities clearly and avoids putting all logic in one file.

Documentation

The README is clear, complete, and useful for running the project.